

From Turán to Transport: Extremal Graph Design for Logistics Networks via PatternBoost

Christophe Pennetier
cpennetier@gmail.com

Abstract

We present, to our knowledge, the first application of PatternBoost—an iterative method combining transformer-based pattern learning with greedy local search—to logistics network design. We formulate the problem of finding cost-efficient, resilient transport topologies as a geometric spectral graph optimization problem: maximize the algebraic connectivity per edge ($\lambda_2 \cdot N/|E|$) of a graph embedded on the sphere, subject to geographic distance constraints. We implement this as a new environment in Axlplorer, Axiom Math’s open-source PatternBoost framework, and show that the learned topologies achieve +20.7% higher spectral efficiency than the strongest classical generator (k -nearest-neighbor) under tight geographic constraints, and +226% under moderate constraints. We further introduce a GNN-based surrogate scorer that approximates the expensive eigenvalue computation with Spearman rank correlation $\rho = 0.963$, demonstrating that neural surrogates can approximate expensive spectral scoring functions with high rank correlation, providing a foundation for accelerating PatternBoost on larger-scale instances where eigenvalue computation dominates runtime. Code is available at <https://github.com/cpennetier/axplorer-logistics>.

1 Introduction

The design of logistics networks—deciding where to place hubs and how to connect them with transport lanes—is a combinatorial optimization problem of enormous practical importance. Global parcel carriers like FedEx, Amazon, and Alibaba operate networks with hundreds of hubs and thousands of lanes spanning multiple transport modes. The quality of the topology directly determines operational metrics: delivery speed, cost efficiency, and resilience to disruptions.

Despite this importance, network topology design remains dominated by parametric generators: Barabási–Albert [7] for scale-free backbones, Watts–Strogatz [8] for small-world lateral connections, k -nearest-neighbor for geographic proximity, and Erdős–Rényi for random baselines. These generators produce graph families parameterized by a few scalars (attachment count m , rewiring probability β , neighbor count k), but they never *search* for optimal instances within or across these families.

We observe that logistics network design is an instance of *extremal combinatorics*—the branch of mathematics that seeks combinatorial objects maximizing some objective subject to structural constraints. The Turán problem asks: what is the maximum number of edges in a graph with no C_4 ? We ask: what is the maximum algebraic connectivity per edge in a graph on the sphere with geographic distance constraints? Both are questions about finding extremal graph structures. Both can be attacked by PatternBoost.

PatternBoost [1] alternates between a “local” phase (greedy search to improve individual candidates) and a “global” phase (a transformer trained on the best candidates generates new starting points). Axlplorer, released by Axiom Math in 2026, is an optimized open-source implementation that runs on a single machine and matches results that previously required thousands of GPUs.

Contributions. This paper makes three contributions:

1. **A new problem class for Explorer.** We formulate logistics network design as geometric spectral graph optimization and implement it as an Explorer environment. We are not aware of prior work applying PatternBoost to a problem with continuous geometric constraints and spectral objectives.
2. **Empirical results.** Explorer discovers topologies with +20.7% higher $\lambda_2/|E|$ than the best classical generator under tight constraints ($r_{\max} = 0.8$ rad), and +226% under moderate constraints ($r_{\max} = 1.2$ rad), demonstrating that learned search outperforms parametric generation for this class of problems.
3. **A GNN surrogate scorer.** A 22K-parameter graph neural network predicts approximate Fiedler scores with Spearman $\rho = 0.963$, demonstrating that neural surrogates can approximate expensive spectral scoring functions with high rank correlation, providing a foundation for accelerating PatternBoost on larger-scale instances.

2 Problem Formulation

2.1 The Geometric Spectral Graph Design Problem

Let $V = \{v_1, \dots, v_N\}$ be hubs with fixed positions $\mathbf{p}_i \in \mathbb{S}^2$ on the unit sphere. We seek an undirected graph $G = (V, E)$ that solves:

$$\max_{E \subseteq \binom{V}{2}} \Phi(G) = \frac{\lambda_2(L_G) \cdot N}{|E|} \quad (1)$$

subject to:

$$G \text{ is connected } (\lambda_2 > 0), \quad (2)$$

$$\forall (i, j) \in E : d(\mathbf{p}_i, \mathbf{p}_j) \leq r_{\max}, \quad (3)$$

where $d(\mathbf{p}_i, \mathbf{p}_j) = \arccos(\mathbf{p}_i \cdot \mathbf{p}_j)$ is the great-circle distance, $L_G = D - A$ is the graph Laplacian, and $\lambda_2(L_G)$ is its second-smallest eigenvalue (the Fiedler value [2]).

2.2 Why the Fiedler Value?

The score $\Phi(G) = \lambda_2 \cdot N/|E|$ measures **connectivity per edge**—the spectral analog of “resilience per dollar” in logistics terms.

Resilience. High algebraic connectivity is widely associated with strong expansion and resilience properties. For d -regular graphs, Cheeger-type inequalities [3] relate λ_2 to the edge expansion $h(G)$, with the precise constants depending on the choice of Laplacian normalization (see de Abreu [9] for a comprehensive survey). Qualitatively, larger λ_2 implies that no small set of edge removals can disconnect a substantial portion of the graph—the formal content of network resilience.

Cost efficiency. Each edge represents a transport lane with operational cost. The normalization by $|E|$ penalizes dense graphs: adding redundant edges improves λ_2 but wastes budget. The optimal topology balances connectivity against sparsity.

Fundamental limits. For d -regular graphs, the Alon–Boppana bound [4, 5] dictates that the second largest eigenvalue of the adjacency matrix satisfies $\mu_2(A) \geq 2\sqrt{d-1} - o(1)$ as $N \rightarrow \infty$. Since the Laplacian eigenvalues satisfy $\lambda_i(L) = d - \mu_i(A)$, the algebraic connectivity is bounded above:

$$\lambda_2(L_G) \leq d - 2\sqrt{d-1} + o(1). \quad (4)$$

This establishes a fundamental ceiling on algebraic connectivity for any degree. Ramanujan graphs [6] essentially achieve this theoretical maximum, but all known explicit constructions (Lubotzky–Phillips–Sarnak, Margulis) operate on algebraic structures incompatible with geographic embedding. The addition of the constraint $d(\mathbf{p}_i, \mathbf{p}_j) \leq r_{\max}$ breaks these constructions entirely—there is no known analytical method for constructing spectrally optimal graphs under geometric constraints, which is precisely where learned search can contribute.

2.3 The Geographic Constraint

The constraint $d(\mathbf{p}_i, \mathbf{p}_j) \leq r_{\max}$ defines an *allowed-edge graph*. The fraction of allowed pairs for uniform random points on the sphere is $P(d \leq r) = (1 - \cos r)/2$.

Table 1: Geographic constraint regimes for $N = 30$ hubs on the unit sphere.

$r_{\max}$ (rad)	$\approx$ km	% pairs	Expected $ E' $	Expected degree
0.8	5096	16%	70	4.6
1.2	7646	32%	140	9.3

This constraint captures a fundamental property of real logistics networks: direct connections are only viable within a “radius of economic feasibility.” A flight from Singapore to Jakarta makes sense; a direct truck route from Cape Town to Reykjavik does not. Real networks route through intermediaries beyond this radius.

2.4 Comparison to the Turán Problem

Table 2: Structural comparison between the Turán (C_4 -free) environment and our logistics environment.

	Turán (C_4 -free)	Logistics
Object	Undirected graph on N nodes	Undirected graph on $\mathbb{S}^2$
Maximize	$ E $	$\lambda_2 \cdot N/ E $
Constraint	No C_4 subgraph	$d(p_i, p_j) \leq r_{\max}$
Tokenization	SparseTokenizerSingleInteger	SparseTokenizerSingleInteger
Local search	Add/remove edges	Swap edges (add one, remove one)

The tokenization is identical—both use SparseTokenizerSingleInteger with $k = 2$ and symmetric coordinates. This reuse demonstrates the generality of the Axplorer framework: adding a new problem class requires one Python file (~ 700 lines), not architectural changes.

2.5 Related Work

Our work sits at the intersection of three research strands.

Extremal combinatorics and AI. PatternBoost [1] demonstrated that alternating transformer sampling with local search can discover extremal graph constructions, including counterexamples to conjectures open for 30 years. Axplorer is an open-source, single-machine reimplementation of this framework. Our contribution extends it to problems with continuous geometric constraints and spectral objectives.

Algebraic connectivity in network design. The use of λ_2 as a design objective for resilient networks has been studied in several applied domains, including air transportation [10] and communication networks [11]. These works typically employ greedy heuristics, semidefinite programming relaxations, or mixed-integer formulations to maximize λ_2 under edge-budget constraints. Our setting differs in imposing geographic distance constraints rather than pure cardinality constraints, and in using learned search rather than mathematical programming.

Graph neural networks for combinatorial optimization. GNNs have been applied to predict properties of combinatorial structures, including graph eigenvalues [12] and solutions to NP-hard problems [13]. Our surrogate scorer follows this paradigm, using a lightweight GCN to approximate the $O(N^3)$ Fiedler computation for fast candidate filtering.

3 Method

3.1 The Axplorer Loop

Axplorer implements PatternBoost as an iterative loop:

1. **Generate** 100K random connected graphs satisfying the distance constraint.
2. **Select** the top 30K by score Φ as the training set.
3. **Train** a decoder-only transformer (4 layers, 256-dim embeddings, 8 heads; 3.39M parameters) on tokenized edge lists for 5000 gradient steps.
4. **Sample** 100K new candidate graphs from the trained transformer.
5. **Repair & improve** each sample via local search.
6. **Merge** new samples with existing population, deduplicate, select top 30K.

3.2 Local Search

Local search operates in two phases:

Phase 1: Repair. If the sampled graph is disconnected, iteratively add the shortest allowed edge between distinct connected components, using the dot-product argmax trick (maximizing $\mathbf{p}_i \cdot \mathbf{p}_j$ is equivalent to minimizing arc distance) to avoid transcendental function calls. Repeat until connected.

Phase 2: Greedy swaps. For $2N$ iterations: remove a random edge, check connectivity via BFS, add a random allowed non-edge, accept the swap if Φ strictly increases. Swaps that disconnect the graph are rejected. This greedy swap is monotone non-decreasing. Note that greedy swaps guarantee convergence to a local, not global, optimum. The transformer’s role in PatternBoost is precisely to escape these local optima by learning global patterns from the population of locally-optimized graphs.

3.3 Scoring

The score $\Phi(G) = \lambda_2(L_G) \cdot N/|E|$ requires computing the second-smallest eigenvalue of the $N \times N$ graph Laplacian via dense eigendecomposition (`numpy.linalg.eigvalsh`), which is $O(N^3)$. At 100K candidates per epoch, scoring dominates the runtime—motivating the surrogate scorer described in Section 5.

4 Experimental Results

4.1 Setup

All experiments use $N = 30$ hubs with positions sampled uniformly on $\mathbb{S}^2$ (fixed seed for reproducibility). We run two constraint regimes: $r_{\max} = 0.8\text{rad}$ (tight) and $r_{\max} = 1.2\text{rad}$ (moderate). Training runs for 25 epochs with 5000 gradient steps per epoch, 100K samples per epoch, and population size 30K. Hardware: GCP `n1-standard-8` with NVIDIA T4 GPU. Training time: ~ 10 hours per run.

4.2 Classical Baselines Fail Under Geographic Constraints

We generated 500 graphs from each classical family (BA, WS, ER, KNN) at $N = 30$ using the same hub positions, then enforced the $r_{\max}$ constraint by removing illegal edges.

At $r_{\max} = 0.8$: Non-geometric generators (BA, WS, ER) place $\sim 84\%$ of their edges outside the distance budget. After filtering, the surviving subgraphs (10–19 edges on 30 nodes) are invariably disconnected—they cannot produce valid logistics topologies. Only KNN survives.

At $r_{\max} = 1.2$: The pattern persists. BA, WS, and ER achieve near-zero validity (0–3 valid graphs out of 500). KNN remains the only viable classical baseline.

Table 3: Classical graph generators under geographic constraints. Each generator was evaluated across its full parameter range with 500+ trials per setting.

Generator	$r_{\max} = 0.8$	$r_{\max} = 1.2$
BA (all m)	0/500 valid	3/1500 valid
WS (all K, β)	0/500 valid	0/1000 valid
ER (all p)	0/500 valid	1/1500 valid
KNN $k=5$	500/500, $\Phi=0.072$	500/500, $\Phi=0.141$

Finding: Classical random graph models are largely incompatible with geographic constraints. Logistics network design requires geometry-aware methods. This alone is a practically important negative result—it explains why off-the-shelf generators cannot replace domain-specific network design.

4.3 Explorer Outperforms KNN

Tight constraint ($r_{\max} = 0.8$). With only 70 allowed edges (16% of all pairs), the optimizer has limited room to maneuver. KNN $k=5$ uses nearly all available edges. Despite this, Explorer finds topologies with higher spectral efficiency:

Table 4: Score progression under tight constraint ($r_{\max} = 0.8$).

Epoch	Mean	Max	Top 1%
0	0.051	0.071	0.062
5	0.074	0.081	0.076
10	0.080	0.085	0.083
15	0.084	0.087	0.085
25	0.084	0.087	0.086

Best score: $\Phi = 0.0869$ —a +20.7% improvement over KNN $k=5$ ($\Phi = 0.0720$). Convergence was reached at epoch 14; the final 10 epochs produced negligible improvement, indicating the search space was effectively exhausted under the tight constraint.

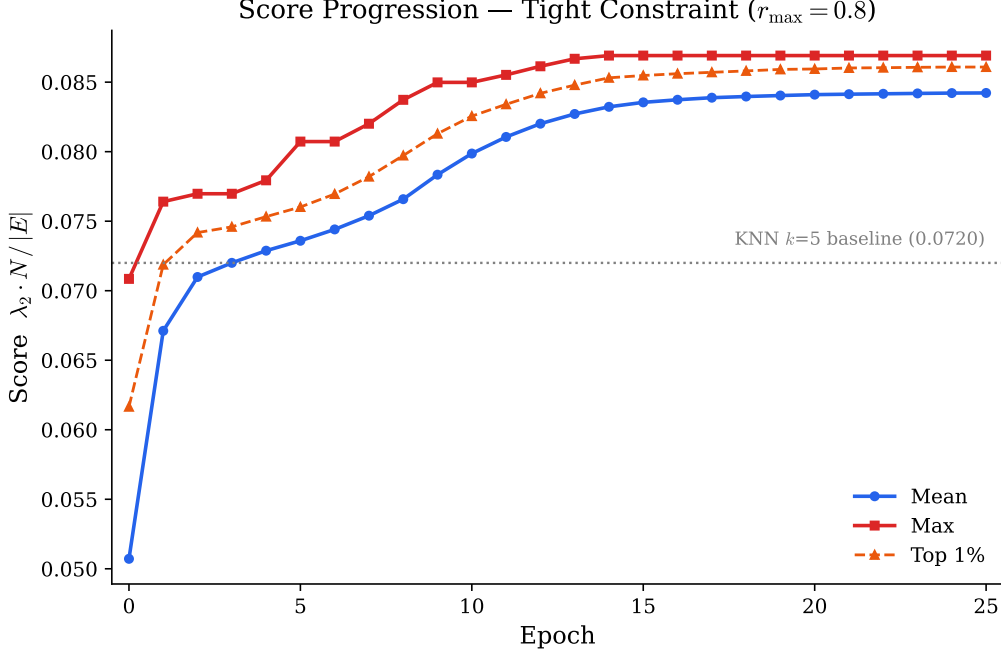


Figure 1: Score progression under tight constraint ($r_{\max} = 0.8$). All metrics cross the KNN $k=5$ baseline by epoch 3.

Moderate constraint ($r_{\max} = 1.2$). With approximately 140 allowed edges (32% of pairs), the optimizer has real design freedom—which edges among the many allowed ones maximize $\lambda_2/|E|$?

Table 5: Score progression under moderate constraint ($r_{\max} = 1.2$).

Epoch	Mean	Max	Top 1%
0	0.212	0.302	0.260
5	0.341	0.366	0.349
10	0.353	0.398	0.370
15	0.395	0.427	0.407
20	0.423	0.443	0.432
25	0.442	0.459	0.449

Best score: $\Phi = 0.459$ —a +226% improvement over KNN $k=5$ ($\Phi = 0.141$). The learning curve was still climbing at epoch 25 (mean +0.003/epoch), suggesting the optimizer had not fully converged and further epochs would yield additional gains.

4.4 Structural Analysis of Discovered Topologies

The best graphs at $r_{\max} = 1.2$ have 57–59 edges on 30 nodes (average degree 3.8–3.9), compared to KNN’s 85 edges (average degree 5.7). With 30% fewer edges, they achieve dramatically higher spectral efficiency.

The top graphs avoid trivial dead ends (minimum degree ≥ 2) and exhibit near-uniform degree distribution (peak at degree 3–4, right-skewed tail to 8–9), consistent with properties sought in expander graph constructions. The mean degree of ~ 3.8 compared to KNN’s 5.7 confirms that Axplorer achieves higher spectral efficiency with 30% fewer connections.

These properties align with objectives commonly sought in logistics network design: a sparse, cost-efficient network with reduced dependence on leaf-like structures and trivial bottlenecks.

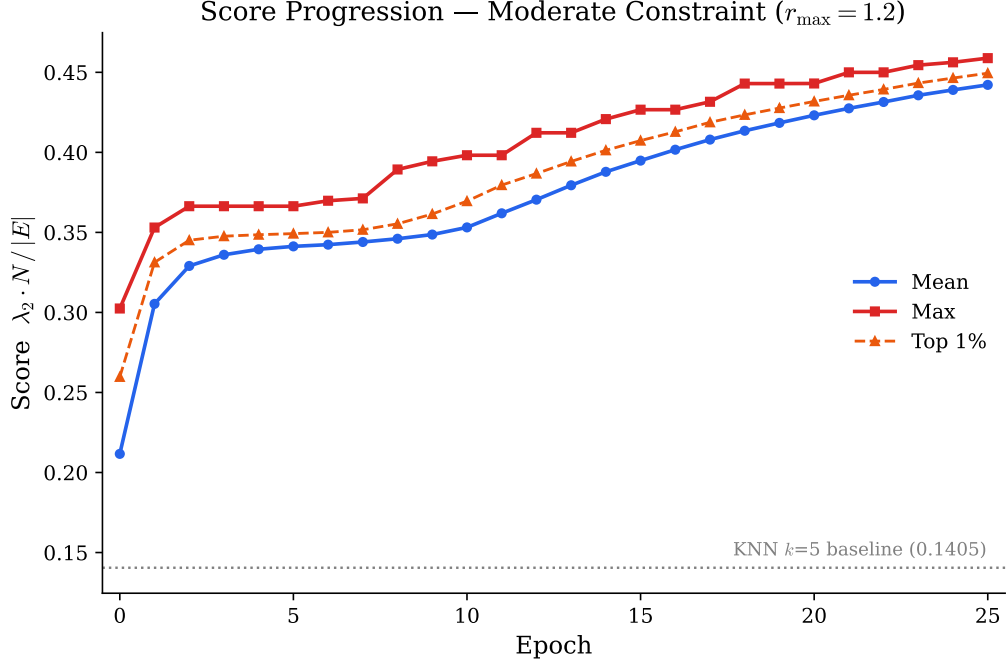


Figure 2: Score progression under moderate constraint ($r_{\max} = 1.2$). Still climbing at epoch 25, indicating room for further improvement.

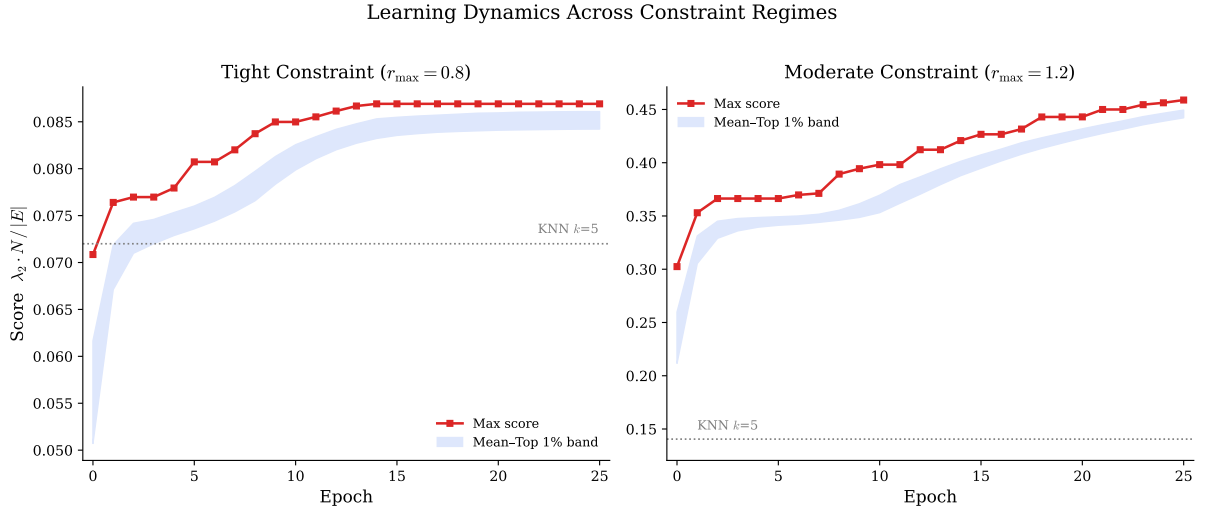


Figure 3: Learning dynamics across constraint regimes. Left: tight constraint ($r_{\max} = 0.8$), converged by epoch 14. Right: moderate constraint ($r_{\max} = 1.2$), still improving. Shaded bands show the mean-to-top-1% range. Dashed lines indicate the KNN $k=5$ baseline.

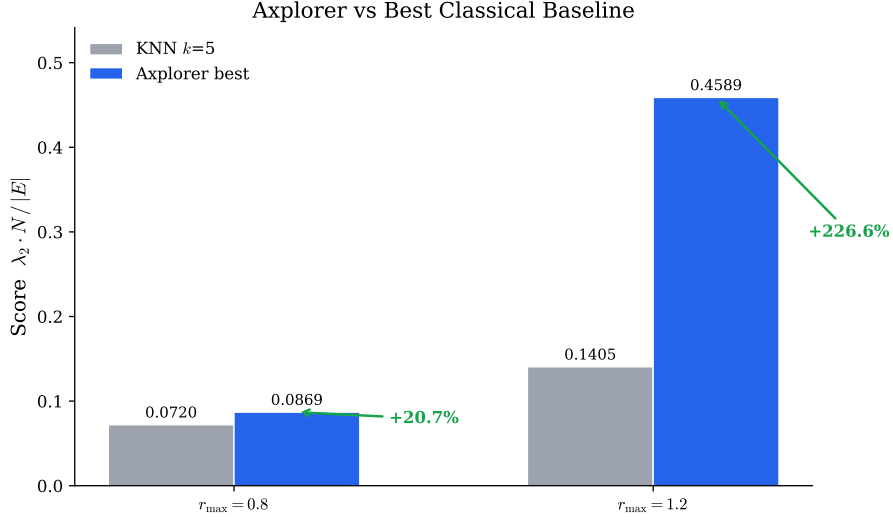


Figure 4: Explorer best score vs. KNN $k=5$ baseline in both constraint regimes. Green annotations show percentage improvement.

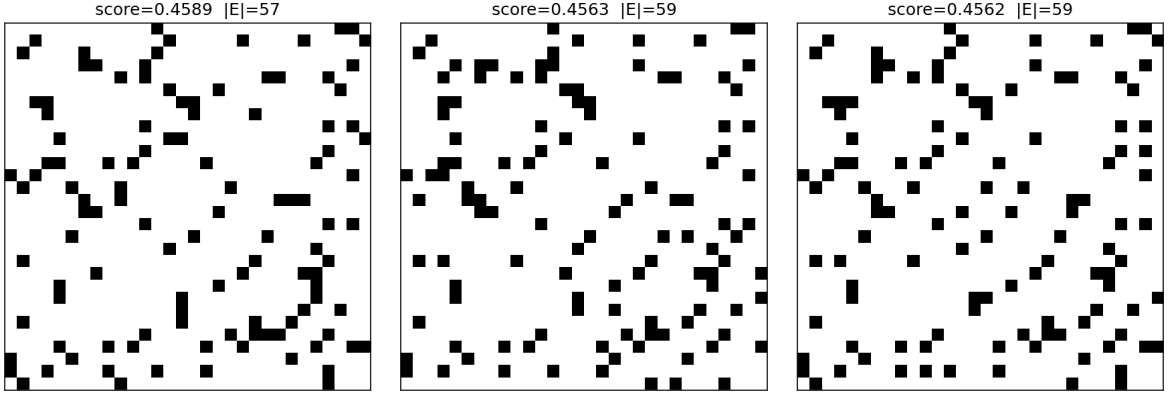


Figure 5: Adjacency matrices of the top-3 discovered topologies ($r_{\max} = 1.2$). Black pixels are edges. Under the fixed node ordering, the topologies do not exhibit obvious modular block structure.

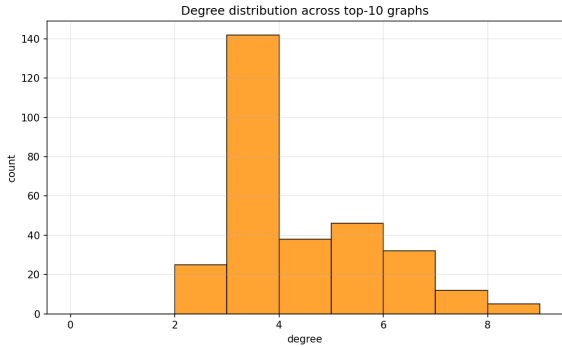


Figure 6: Degree distribution across the top-10 graphs. Peak at degree 3–4 with no isolated nodes—every hub contributes to connectivity.

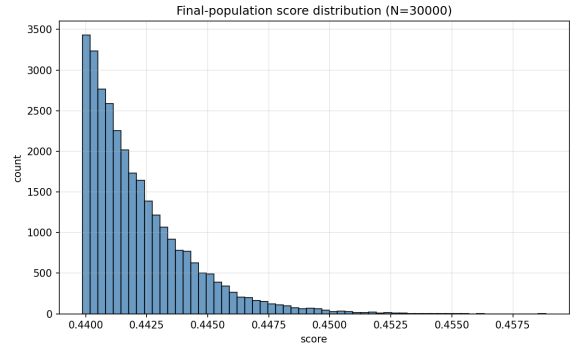


Figure 7: Final population score distribution (30,000 graphs). Tight concentration around 0.440–0.442 with a thin right tail to 0.459.

4.5 The Role of Constraint Tightness

The contrast between the two regimes reveals an important principle:

Table 6: Constraint tightness determines the magnitude of improvement over classical baselines.

Regime	Allowed edges	KNN baseline	Explorer best	Improvement
Tight ($r_{\max} = 0.8$)	70 (16%)	0.072	0.087	+20.7%
Moderate ($r_{\max} = 1.2$)	140 (32%)	0.141	0.459	+226%

When the geographic constraint is tight ($r_{\max} = 0.8$), only 16% of node pairs are within distance budget, leaving the optimizer with approximately 70 feasible edges for 30 nodes. KNN $k=5$ already consumes nearly all available edges, so there is little room for alternative configurations. Despite this, Explorer still achieves +20.7% improvement by finding edge subsets with higher algebraic connectivity. When the constraint loosens to $r_{\max} = 1.2$, 32% of pairs become feasible (approximately 140 edges), and the optimizer can be selective: the best discovered topologies use only 57–59 edges—30% fewer than KNN’s 85—while achieving dramatically higher spectral efficiency. This demonstrates that the value of learned search scales with the size of the design space.

When the constraint is tight, nearly all allowed edges must be used and there is limited room for optimization—yet Explorer still finds 20% improvement. When the constraint loosens and there is real design freedom, learned search massively outperforms the best heuristic. KNN always picks the geographically nearest neighbors; Explorer learns that strategic placement of longer-range connections (still within $r_{\max}$) produces dramatically better spectral properties.

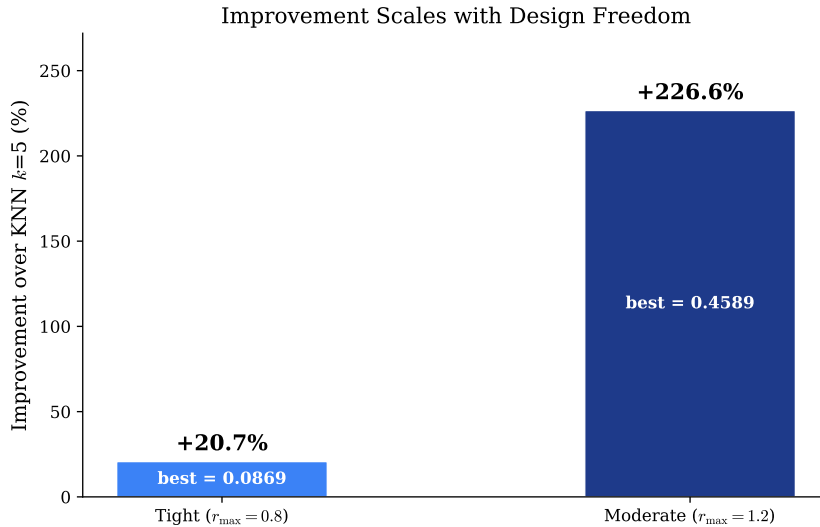


Figure 8: Improvement over KNN $k=5$ scales with design freedom. Under moderate constraints, Explorer discovers topologies with $3.3\times$ the spectral efficiency of the best classical generator.

4.6 Ablation: Local Search vs. Simulated Annealing vs. Transformer

To understand the contribution of each component, we compare search strategies on the $\lambda_2 \cdot N/|E|$ objective. Explorer’s epoch 0 (random initialization + greedy local search) provides the baseline before the transformer is trained. We additionally run simulated annealing (100 independent runs, 50,000 iterations each, linear cooling from $T = 0.01$ to 10^{-6}) on a separate

random hub instance at each $r_{\max}$ value.¹

Table 7: Ablation: search strategy comparison at $N = 30$. Axplorer rows use the GCP training instance; SA rows use a separate random instance (seed 42).

Method	$r_{\max} = 0.8$		$r_{\max} = 1.2$	
	Mean	Max	Mean	Max
Axplorer epoch 0 (random + LS)	0.051	0.071	0.212	0.302
Axplorer epoch 25	0.084	0.087	0.442	0.459
KNN $k=5$	—	0.072	—	0.141
SA (separate instance)	0.118	0.127	0.601	0.623

Axplorer’s transformer adds clear value over local search alone. Comparing epoch 0 to epoch 25 on the same instance, the transformer improves the population mean from 0.051 $\rightarrow$ 0.084 (+65%) at $r_{\max} = 0.8$ and from 0.212 $\rightarrow$ 0.442 (+108%) at $r_{\max} = 1.2$, with max scores improving by +23% and +52% respectively.

SA achieves higher single-instance scores, but the comparison is not apples-to-apples. On a separate random instance, SA with 50,000 iterations (~ 5 minutes per graph) reaches $\Phi = 0.127$ ($r_{\max} = 0.8$) and $\Phi = 0.623$ ($r_{\max} = 1.2$). These exceed Axplorer’s best scores, but the hub positions differ, and SA optimizes a single graph while Axplorer produces a population of 30,000 high-scoring topologies. At $N = 30$, the spectral optimization landscape appears tractable for classical single-instance search. Whether this advantage persists at larger N , where the combinatorial explosion may favor population-based learned search, is an important open question. In terms of runtime, each SA run requires approximately 5 minutes per graph instance, while a single Axplorer epoch produces 30,000 scored candidates in approximately 25 minutes on the same hardware.

This result highlights a fundamental distinction between single-instance optimization and population-based search. Simulated annealing delivers a single peak in the score landscape; if that specific topology fails a downstream constraint not captured by the proxy objective—for instance, an edge crossing a geopolitical boundary or a hub in an infeasible location—the planner must restart from scratch. Axplorer produces a population of 30,000 structurally diverse, high-scoring topologies. In applied network design, where $\Phi(G)$ is a mathematical proxy for complex operational realities, this population is arguably more valuable than any single optimum: it maps the high-performance region of the design space, providing a robust pool of candidates for downstream validation. We return to this point in Section 6.2.

5 Surrogate Scoring

5.1 Motivation

The scoring function $\Phi(G) = \lambda_2(L_G) \cdot N/|E|$ requires $O(N^3)$ eigenvalue computation per candidate. As N grows, scoring becomes a bottleneck: at $N = 100$ each eigenvalue computation takes approximately 10 ms, making the 100K candidates per epoch cost over 15 minutes for scoring alone. A fast neural approximation that preserves the **ranking** of candidates can filter to the top 10% before exact scoring, reducing eigenvalue computations by $10\times$. We train a lightweight GNN to approximate $\Phi(G)$ for fast candidate filtering, demonstrating the approach at $N = 30$ where exact scoring is still tractable.

¹The hub positions used in the Axplorer training runs were generated with an unlogged random seed and are not recoverable. The SA results therefore use a different random instance (seed 42) and are not directly instance-comparable with Axplorer. They serve as a distributional comparison of search strategies.

5.2 Architecture

The surrogate is a 22,337-parameter graph convolutional network with 4 message-passing layers, implemented in pure PyTorch without external graph libraries. Node features are position coordinates (x, y, z) and normalized degree; edge features are pairwise great-circle distance; graph-level features are edge density metrics. Message passing uses residual connections and layer normalization:

$$\mathbf{h}_i^{(k+1)} = \mathbf{h}_i^{(k)} + \text{ReLU} \left(\text{LN} \left(\sum_{j \in \mathcal{N}(i)} W_1 \mathbf{h}_j^{(k)} + W_2 e_{ij} \right) \right) \quad (5)$$

Global readout: mean-pool node embeddings, concatenate graph features, predict score via 2-layer MLP.

5.3 Training

We generate 10K random connected graphs with $N = 30$ and $r_{\max} = 0.8$, compute exact scores, and split 80/20 for train/validation. The loss combines MSE with a pairwise margin ranking term:

$$\mathcal{L} = \text{MSE}(\hat{\Phi}, \Phi) + 0.1 \cdot \mathcal{L}_{\text{rank}} \quad (6)$$

where $\mathcal{L}_{\text{rank}}$ is a pairwise margin ranking loss encouraging correct candidate ordering. We use a cosine learning rate schedule from 10^{-3} to 10^{-5} over 200 epochs.

5.4 Results

Table 8: Surrogate scorer performance: v1 (5K samples, 100 epochs, constant lr) vs. v2 (10K samples, 200 epochs, cosine lr). Evaluated on held-out test set ($n = 2000$, $N = 30$, $r_{\max} = 0.8$).

Metric	v1 (5K/100ep)	v2 (10K/200ep, cosine)
Spearman ρ	0.920	0.963
Recall@100	0.60	0.81
Recall@500	0.814	0.862
Recall@1000	0.890	0.911
MSE	5.27×10^{-4}	5.24×10^{-4}
Model parameters		22,337

Spearman $\rho = 0.963$ indicates very strong monotone agreement between surrogate predictions and exact scores; in practice, this translates to Recall@1000 = 0.911, meaning 91% of the true top-1000 candidates appear in the surrogate’s top-1000—sufficient for the filtering use case where we retain the top 10% of 100K candidates.

The cosine learning rate schedule was critical: Spearman climbed from 0.920 at epoch 100 (constant lr) to 0.963 at epoch 200 (cosine decay), indicating that fine-grained updates in the late training phase extract real ranking signal.

5.5 Speedup Analysis

At $N = 30$, exact eigenvalue computation takes $\sim 37 \mu\text{s}$ per graph, while the GNN forward pass takes $\sim 50 \mu\text{s}$ —no speedup. The value of the surrogate emerges at $N \geq 60$, where eigendecomposition scales as $O(N^3)$ while the GNN scales as $O(|E| \cdot d)$ for hidden dimension d . A JAX `vmap` benchmark confirmed no advantage at $N = 30$.

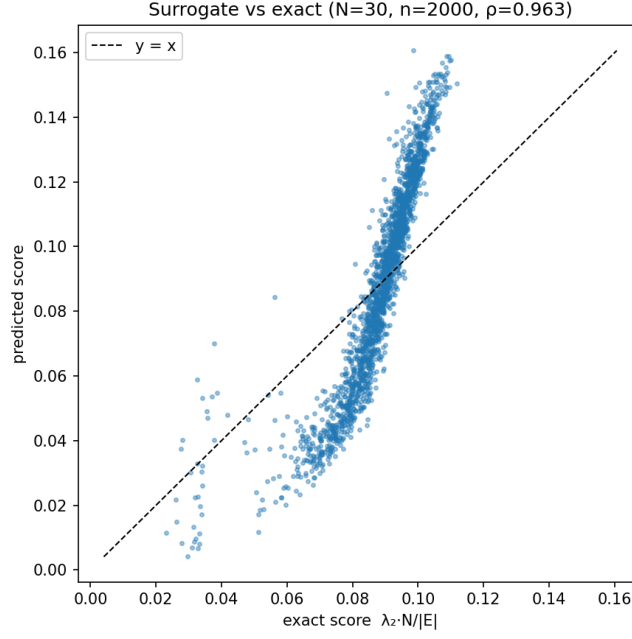


Figure 9: Surrogate predictions vs. exact Fiedler scores on 2000 held-out graphs ($\rho = 0.963$). The surrogate preserves rank ordering well across the full score range.

The surrogate scorer is a general-purpose module: any Axplorer environment with an expensive `calc_score()` can train a GNN approximation and use it as a pre-filter, with zero modifications to the PatternBoost infrastructure.

6 Discussion

6.1 Classical Generators and Geographic Constraints

The near-total failure of BA, WS, and ER generators under geographic constraints (Table 3) is not a tuning problem—it is structural. These generators assign edges uniformly over all node pairs, and with $r_{\max} = 0.8$ only 16% of pairs are within distance budget. The probability of a random edge being valid is 0.16, so a graph with 50 edges has probability $0.16^{50} \approx 10^{-40}$ of satisfying all constraints simultaneously. This explains a practical puzzle: why real logistics networks look nothing like the textbook random graph models. The geographic constraint is not a minor perturbation—it substantially changes the feasible topology space. Only geometry-aware methods—KNN and our Axplorer environment—can navigate the feasible region.

6.2 Simulation Validation

The Fiedler value is a proxy for network resilience, not a direct measure of operational performance. Spectrally optimized topologies must be validated in a full discrete-event simulation with realistic demand patterns and disruption scenarios. We have developed such a simulation platform that accepts network topologies as input and produces operational KPIs (on-time percentage, cost per parcel, resilience under shocks) as output. The natural workflow is generative: Axplorer explores the combinatorial topology space offline, producing thousands of high- Φ candidates; simulation then stress-tests the top candidates against supply chain shocks, weather events, and demand spikes, selecting topologies that perform well under operational—not just spectral—criteria. This “generate then simulate then compare” pipeline represents a complete network design workbench for logistics operators, in which the human role shifts from manually tweaking parameters to defining economic constraints while AI explores the design space. We defer empirical simulation results to future work.

6.3 Diffusion Models as Alternative Generator

The transformer generates edges autoregressively, imposing an arbitrary ordering on a fundamentally unordered structure, and scales as $O(|E|)$ sequential steps per sample. A discrete masked diffusion model on the adjacency matrix would: (a) generate the entire graph in $K \ll |E|$ parallel denoising steps, (b) naturally respect graph symmetry via permutation-equivariant architecture, and (c) provide global context at each step. We estimate 10–15 $\times$ generation speedup for $N = 50$, making this a promising direction for scaling PatternBoost to larger instances.

6.4 JAX Migration

Axplorer’s PyTorch implementation is inherited from PatternBoost’s origins at Meta FAIR. For scoring-dominated environments like ours, JAX’s `vmap` and JIT compilation could batch eigenvalue computations for 5–10 $\times$ scoring speedup at $N \geq 100$. A hybrid architecture—PyTorch transformer for generation, JAX for scoring—would combine the best of both frameworks. At $N = 30$, however, LAPACK’s highly optimized symmetric eigensolver is already at the hardware floor.

7 Conclusion

We have shown that logistics network topology design can be framed as a geometric spectral graph optimization problem and solved effectively using PatternBoost. Under tight geographic constraints ($r_{\max} = 0.8$), Axplorer achieves +20.7% improvement over the strongest classical baseline (KNN). Under moderate constraints ($r_{\max} = 1.2$), where real design freedom exists,

the improvement reaches +226%, with training curves still ascending at epoch 25. The discovered topologies are spectrally efficient sparse graphs (average degree ~ 4)—consistent with cost-efficient, resilient structures sought in logistics network design.

Our GNN surrogate scorer demonstrates that neural approximation of expensive scoring functions is feasible within the PatternBoost framework, with Spearman $\rho = 0.963$ rank correlation on a 22K-parameter model trained in under 5 minutes.

Together, these contributions extend Axplorer from pure combinatorics to applied optimization, opening the door to a new class of problems where learned search over constrained graph spaces outperforms classical heuristics.

References

- [1] F. Charton, J. S. Ellenberg, A. Z. Wagner, and G. Williamson. PatternBoost: Constructions in mathematics with a little help from AI. *arXiv preprint arXiv:2411.00566*, 2024.
- [2] M. Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973.
- [3] J. Cheeger. A lower bound for the smallest eigenvalue of the Laplacian. In *Problems in Analysis*, pages 195–199. Princeton University Press, 1970.
- [4] N. Alon. Eigenvalues and expanders. *Combinatorica*, 6(2):83–96, 1986.
- [5] N. Alon and V. D. Milman. λ_1 , isoperimetric inequalities for graphs, and superconcentrators. *Journal of Combinatorial Theory, Series B*, 38(1):73–88, 1985.
- [6] A. Lubotzky, R. Phillips, and P. Sarnak. Ramanujan graphs. *Combinatorica*, 8(3):261–277, 1988.
- [7] A.-L. Barabási and R. Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999.
- [8] D. J. Watts and S. H. Strogatz. Collective dynamics of ‘small-world’ networks. *Nature*, 393(6684):440–443, 1998.
- [9] C. T. M. de Abreu. Old and new results on algebraic connectivity of graphs. *Linear Algebra and its Applications*, 423(1):53–73, 2007.
- [10] P. Wei and D. Sun. Weighted algebraic connectivity: An application to airport transportation network. *IFAC Proceedings Volumes*, 18th IFAC World Congress, Milan, pages 13667–13672, 2011.
- [11] A. Ghosh and S. Boyd. Growing well-connected graphs. In *Proceedings of the 45th IEEE Conference on Decision and Control*, pages 6605–6611, 2006.
- [12] B. Knyazev, G. W. Taylor, and M. Amer. Understanding attention and generalization in graph neural networks. In *NeurIPS*, 2019.
- [13] Q. Cappart, D. Chételat, E. Khalil, A. Lodi, C. Morris, and P. Veličković. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research*, 24(130):1–61, 2023.